

Project 2: Multi-Agent Search - My Small Report

By Ludovico Grabau

Question 1: Reflex Agent

How does your agent fare? It will likely often die with 2 ghosts on the default board.

My agent actually performs really well on the default board, even with two ghosts. The evaluation function strikes a good balance it penalizes being too close to active ghosts, rewards chasing scared ghosts, and encourages collecting food when it's safe. By focusing on survival, efficient food gathering, and taking opportunities to hunt ghosts when they're vulnerable, the agent makes smart decisions that give it a solid chance of winning. It often survives and completes the game, showing that a well-thought-out evaluation function can make a big difference, even in tough scenarios like this.

Explain in the report the thinking process that you have followed to improve ReflexAgent.

To improve the ReflexAgent, I focused on designing an evaluation function that aligns with Pacman's main objectives: avoiding ghosts, eating food, and maximizing the score. The function emphasizes survival by penalizing Pacman for getting too close to active ghosts while rewarding it for pursuing scared ghosts it can eat. To encourage efficient food collection, Pacman is incentivized to move closer to the nearest food pellet and slightly penalized for wandering without eating, reducing aimless movement. The evaluation function also adapts dynamically based on the game's current score, striking a balance between immediate rewards and long-term survival. This approach helps Pacman make smarter, more purposeful decisions, allowing it to effectively avoid danger, gather food, and capitalize on opportunities to eat scared ghosts. Overall, the improvements make the ReflexAgent perform significantly better, even in challenging situations with multiple ghosts on the default board.

Question 2: Minimax

Describe briefly in the report how your minimaxAgent works.

The MinimaxAgent uses the minimax algorithm to decide the best action for Pacman by simulating potential game outcomes up to a set depth. Pacman acts as the MAX player, aiming to maximize the score, while the ghosts are treated as MIN players, trying to minimize Pacman's score. The algorithm alternates between Pacman's maximizing moves and the ghosts' minimizing responses, exploring the game tree recursively. It stops at terminal states (win or loss), when the depth limit is reached, or when no legal actions remain. The agent evaluates Pacman's possible moves and selects the one that yields the highest score, ensuring it makes the optimal choice against the ghosts' most challenging strategies.

Make sure you understand why Pacman rushes the closest ghost in this case.

```
python pacman.py -p MinimaxAgent -l trappedClassic -a depth=3
```

In the trappedClassic layout, when running the MinimaxAgent with a depth limit of 3, Pacman rushes toward the closest ghost because the algorithm focuses only on the immediate outcomes within that depth. Minimax assumes the ghosts will play optimally to minimize Pacman's score, and with the limited lookahead, it doesn't account for the longer-term consequences of Pacman's actions. When Pacman is surrounded with no clear escape, the algorithm evaluates all paths as eventually leading to capture. Since staying alive incurs a constant living penalty, Pacman chooses to minimize that penalty by ending the game quickly rushing toward the ghost to force a loss. This behavior highlights the limitations of the minimax algorithm when depth is restricted, as it prioritizes short-term evaluations over long-term survival strategies.

Question 3: Alpha-Beta Pruning

Can you explain in your own words what the alpha-beta pruning method is doing?

Alpha-beta pruning is a technique that makes the minimax algorithm more efficient by skipping parts of the game tree that won't affect the final decision. It uses two bounds alpha (the best score MAX is guaranteed so far) and beta (the best score MIN is guaranteed so far) to decide when it's safe to stop evaluating certain branches. If a branch is found that can't possibly improve the outcome for the current player (MAX or MIN), it's "pruned," or ignored, because it won't influence the final result. For example, at a MAX node, if a branch's value is already greater than beta, MIN would never choose it, so there's no need to explore further. Similarly, at a MIN node, if a branch's value is less than alpha, MAX won't allow it. By avoiding unnecessary calculations, alpha-beta pruning speeds up the process while still ensuring the best possible decision is made.

Question 4: Expectimax

Investigate the results of these two scenarios:

```
python pacman.py -p AlphaBetaAgent -l trappedClassic -a depth=3 -q -n 10
```

Average Score: -501.0, Win Rate: 0/10 (0.00)

```
python pacman.py -p ExpectimaxAgent -l trappedClassic -a depth=3 -q -n 10
```

Average Score: 118.4, Win Rate: 6/10 (0.60)

Make sure you understand why the behavior here differs from the minimax case and write a short explanation in the report.

The difference in performance between the AlphaBetaAgent and the ExpectimaxAgent on the trappedClassic layout comes down to their decision-making strategies. The AlphaBetaAgent, using the minimax algorithm, assumes that the ghosts will always act in the worst possible way for Pacman, trying to minimize his score. This overly cautious approach leads the agent to make overly defensive choices, avoiding opportunities for gains because it always expects the worst. As a result, it performs poorly and never wins.

On the other hand, the ExpectimaxAgent takes a more flexible approach. Instead of assuming the ghosts are adversarial, it treats their actions as probabilistic, calculating the expected outcomes based on the likelihood of different moves. This allows the ExpectimaxAgent to take smarter risks, capitalizing on situations where the ghosts don't behave perfectly. This approach often leads to better results, giving the ExpectimaxAgent a win rate of about 50%. The key takeaway is that the assumptions an agent makes about its environment whether adversarial or probabilistic can have a huge impact on its behavior and success.

Question 5: Evaluation Function

What is the advantage of evaluating states rather than actions in this case?

Evaluating states instead of actions gives the agent a bigger-picture view of the game, which helps it make smarter, more strategic decisions. When evaluating states, the agent can factor in a variety of elements like the distance to food, ghost positions, scared timers, capsules, and the overall game score rather than just looking at the immediate outcome of a single action. This approach works well with algorithms like minimax or alpha-beta pruning, as it allows the agent to think several steps ahead, balancing short-term rewards with long-term survival and goals. By focusing on the overall desirability of a game state, the agent avoids making narrow, short-sighted decisions, like grabbing food while ignoring a lurking ghost, and instead picks actions that are more likely to lead to overall success.